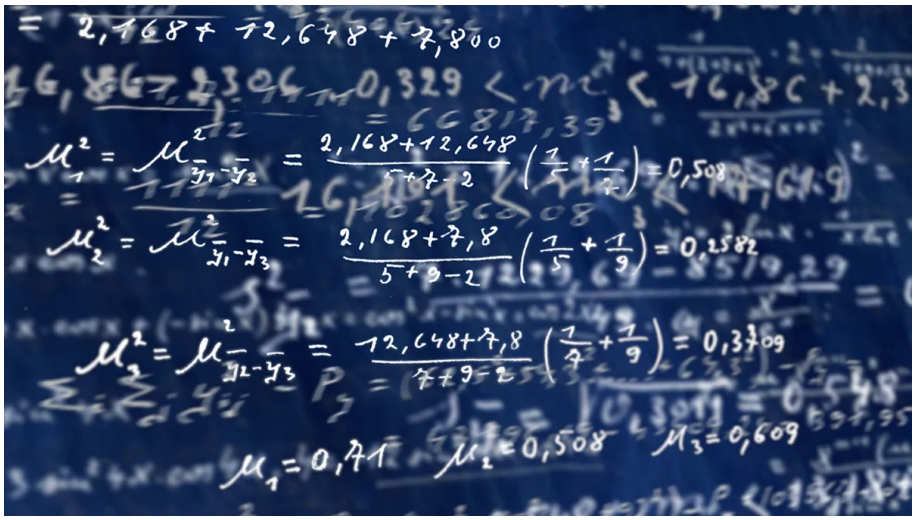
Memoria del Proyecto - Trading Dashboard

Universidad de Navarra (UNAV)



Autor: Jorge Santiago Rodríguez Peñalosa

Fecha: December 29, 2024

Proyecto desarrollado como parte de la asignatura *Python para Análisis de Datos*.

Índice

1 Introducción **2**

2 Enfoque del proyecto y decisiones tomadas **2**

3 Estructura del código y ficheros **2**

4 Descripción detallada de los puntos evaluados **3**
4.1 Lectura y representación del movimiento del par de monedas 3
4.1.1 Descarga de datos 3
4.1.2 Graficación de cotizaciones 4
4.2 Indicadores técnicos 4
4.2.1 Bandas de Bollinger 4
4.2.2 RSI y MACD 5

5 Capturas de pantalla **6**

6 Testeo y cobertura **8**

7 Ejecución del código **9**
7.1 Resumen 10

1. Introducción

En esta memoria se documenta el desarrollo completo de un Trading Dashboard que utiliza la API de Kraken para obtener datos de mercado y proporciona herramientas avanzadas de análisis técnico. El proyecto se centra en la implementación de indicadores como Bandas de Bollinger, RSI y MACD, y en la creación de gráficos interactivos para la visualización de datos.

2. Enfoque del proyecto y decisiones tomadas

El proyecto fue desarrollado en varias etapas:

1. Descarga de datos financieros a través de la API de Kraken utilizando la biblioteca `krakenex`.
2. Implementación de indicadores técnicos como Bandas de Bollinger, RSI y MACD mediante código personalizado.
3. Creación de un dashboard interactivo utilizando Dash para visualizar cotizaciones y generar señales de compra y venta.

Se optó por `krakenex` para garantizar una conexión robusta y en tiempo real con la API de Kraken. Dash fue seleccionado por su capacidad de crear aplicaciones web interactivas de forma sencilla y eficiente.

3. Estructura del código y ficheros

El código está dividido en varios archivos para facilitar su mantenimiento y extensión. A continuación, se describen los principales ficheros y directorios:

- **main.py**: Archivo principal que gestiona la aplicación Dash y la interacción con las otras clases.
- **kraken_data.py**: Contiene la clase `KrakenData`, que maneja la conexión a la API de Kraken para descargar datos de los pares de activos y series temporales.
- **indicator_calculator.py**: Implementa la clase `IndicatorCalculator`, encargada del cálculo de indicadores técnicos como Bandas de Bollinger, RSI y MACD.

A continuación, se presenta un fragmento del código principal `main.py`:

```
import dash
from dash import dcc, html, Input, Output
import dash_bootstrap_components as dbc
import webbrowser
from kraken_data import KrakenData
from indicator_calculator import IndicatorCalculator
import plotly.graph_objects as go

# Inicializar KrakenData
...
```

Este diseño modular permite:

- Reutilizar funciones específicas en diferentes contextos.
- Mantener el código más limpio y fácil de depurar.
- Incorporar nuevos indicadores y funcionalidades sin afectar la estructura base.

4. Descripción detallada de los puntos evaluados

4.1. Lectura y representación del movimiento del par de monedas

4.1.1 Descarga de datos

La descarga de datos se realizó con la clase `KrakenData`, que utiliza la biblioteca `krakenex` para conectarse a la API de Kraken. A continuación, se presenta el código correspondiente:

Listing 1: Clase KrakenData para descarga de datos.

```
class KrakenData:
    def __init__(self):
        self.api = krakenex.API()

    def fetch_asset_pairs(self):
        try:
            response = self.api.query_public('AssetPairs')
            if 'result' not in response:
                raise ValueError("No result field in Kraken API response.")
            pairs = list(response['result'].keys())
            formatted_pairs = [f"{pair[:3]}/{pair[3:]}" for pair in pairs]
            return formatted_pairs
        except Exception as e:
            print(f"Error fetching asset pairs: {e}")
            return []

    def fetch_data(self, pair, interval):
        try:
            response = self.api.query_public('OHLC', {'pair': pair, 'interval': interval})
            if 'error' in response and response['error']:
                raise ValueError(f"Error from Kraken API: {response['error']}")
            data = response['result'][list(response['result'].keys())[0]]
            df = pd.DataFrame(data, columns=['time', 'open', 'high', 'low', 'close', 'vwap', 'volume', 'count'])
            df['time'] = pd.to_datetime(df['time'], unit='s')
            df[['open', 'high', 'low', 'close', 'volume']] = df[['open', 'high', 'low', 'close', 'volume']].astype(float)
```

```
            return df
        except KeyError as e:
            print(f"KeyError: {e}")
            return None
        except Exception as e:
            print(f"An unexpected error occurred: {e}")
            return None
```

4.1.2 Graficación de cotizaciones

Las cotizaciones se graficaron con `plotly`. La aplicación permite al usuario seleccionar el par de monedas a visualizar mediante un menú desplegable.

Listing 2: Código para graficación de cotizaciones.

```
app.layout = dbc.Container([
    dbc.Row([
        dbc.Col([
            html.H1("Trading Dashboard", className="text-
                center mb-4 text-white"),
            dcc.Dropdown(
                id="pair-dropdown",
                options=[{"label": pair, "value": pair.
                    replace("/", "")} for pair in asset_pairs
                    ],
                placeholder="Select a pair",
                className="mb-3",
                style={"color": "black"}
            ),
```

4.2. Indicadores técnicos

4.2.1 Bandas de Bollinger

Las Bandas de Bollinger son un indicador de trading que mide la volatilidad del mercado y proporciona una gran cantidad de información muy útil para tomar decisiones de compra y venta de activos financieros. El cálculo de las Bandas de Bollinger incluye las señales de compra y venta:

Listing 3: Cálculo de Bandas de Bollinger.

```
class IndicatorCalculator:
    @staticmethod
    def calculate_bollinger_bands(df, window=20):
        if len(df) < window:
            raise ValueError("Not enough data points to
                calculate Bollinger Bands.")
        df['mean'] = df['close'].rolling(window=window).mean
            ()
        df['std'] = df['close'].rolling(window=window).std()
        df['upper_band'] = df['mean'] + 2 * df['std']
        df['lower_band'] = df['mean'] - 2 * df['std']
```

```
        df['buy_signal'] = np.where(df['close'] < df['lower_band'], df['close'], np.nan)
        df['sell_signal'] = np.where(df['close'] > df['upper_band'], df['close'], np.nan)
        return df
```

4.2.2 RSI y MACD

Implementación del cálculo del RSI y MACD para proporcionar indicadores técnicos adicionales.

RSI (Relative Strength Index) El *RSI* (Relative Strength Index) es un indicador tipo oscilador utilizado en análisis técnico que muestra la fuerza del precio mediante la comparación de los movimientos individuales al alza o a la baja de los sucesivos precios de cierre.

MACD (Moving Average Convergence Divergence) El *MACD* (Moving Average Convergence Divergence) es un indicador de análisis técnico que se utiliza para medir el impulso y la dirección de una tendencia.

A continuación, se muestra el código para ambos:

Listing 4: Cálculo del RSI y MACD.

```
    @staticmethod
    def calculate_rsi(df, window=14):
        delta = df['close'].diff()
        gain = np.where(delta > 0, delta, 0)
        loss = np.where(delta < 0, -delta, 0)
        avg_gain = pd.Series(gain).rolling(window=window, min_periods=1).mean()
        avg_loss = pd.Series(loss).rolling(window=window, min_periods=1).mean()
        rs = avg_gain / avg_loss
        df['rsi'] = 100 - (100 / (1 + rs))
        return df

    @staticmethod
    def calculate_macd(df, fast_period=12, slow_period=26, signal_period=9):
        df['ema_fast'] = df['close'].ewm(span=fast_period, adjust=False).mean()
        df['ema_slow'] = df['close'].ewm(span=slow_period, adjust=False).mean()
        df['macd'] = df['ema_fast'] - df['ema_slow']
        df['macd_signal'] = df['macd'].ewm(span=signal_period, adjust=False).mean()
        return df
```

5. Capturas de pantalla

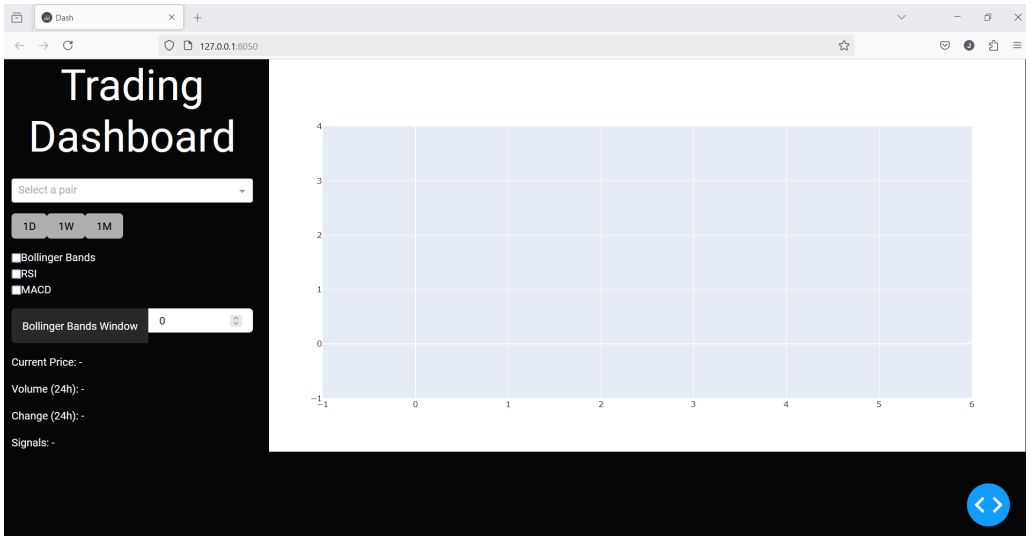


Figure 1: Interfaz principal del dashboard

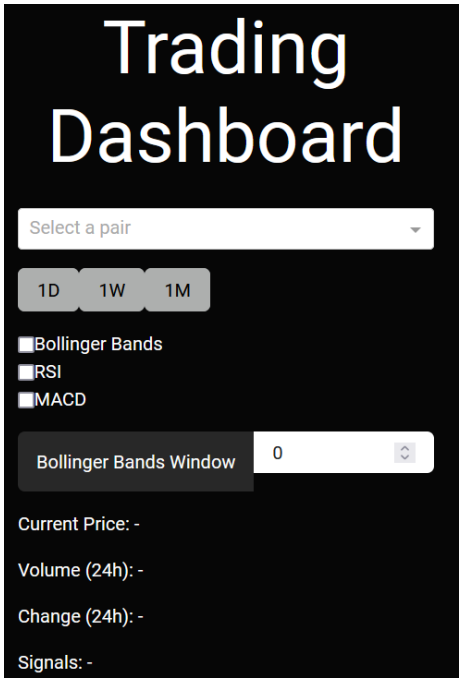


Figure 2: Barra lateral para usuario

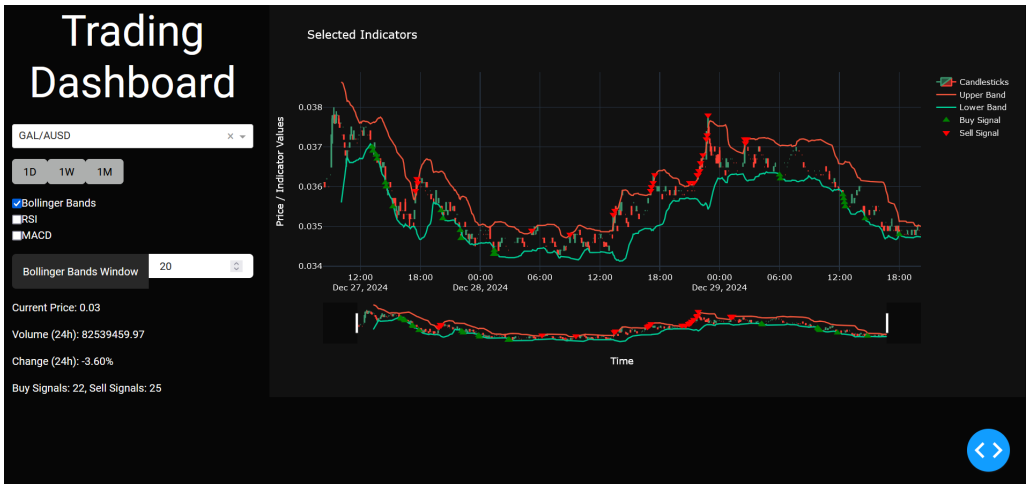


Figure 3: Gráfica con Bandas de Bollinger del par GAL/AUSD

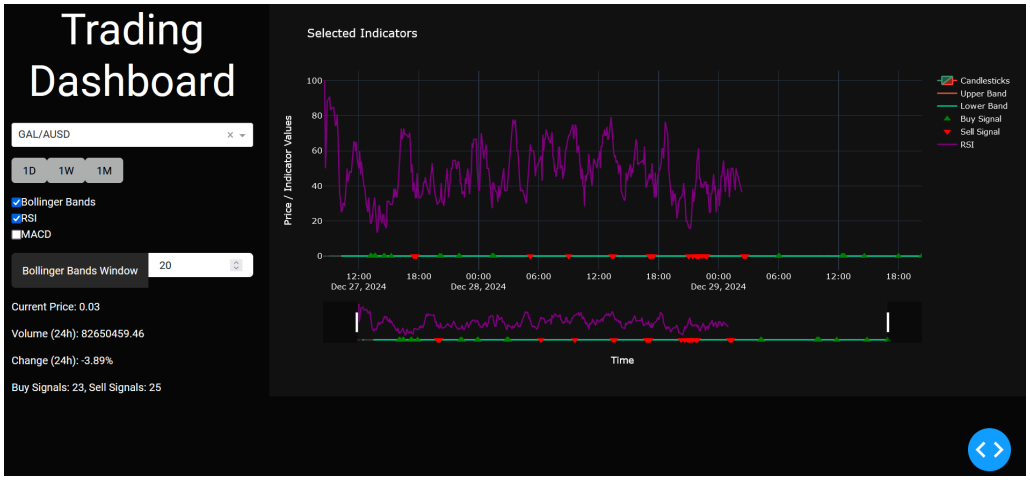


Figure 4: Gráfica con RSI del par GAL/AUSD

- **Current Price**: 0.03 - El precio actual del activo
- **Volume(24h)**: 82650459.46 - Volumen de transacciones
- **Change(24h)**: -3.89% - El precio ha disminuido ese %
- **Buy Signals**: 23 - Señales de compra
- **Sell Signals**: 25 - Señales de venta

6. Testeo y cobertura

Se desarrollaron pruebas unitarias con *unittest* para garantizar la funcionalidad:

Listing 5: Prueba unitaria para KrakenData.

```
import unittest
from ProyectoTradingDashboard import KrakenData
import pandas as pd
class TestKrakenData(unittest.TestCase):

    def setUp(self):
        self.kraken_data = KrakenData()

    def test_fetch_asset_pairs(self):
        pairs = self.kraken_data.fetch_asset_pairs()
        self.assertIsInstance(pairs, list, "The result should
            be a list of asset pairs.")
        self.assertGreater(len(pairs), 0, "The list of asset
           pairs should not be empty.")

    def test_fetch_data(self):
        df = self.kraken_data.fetch_data('ETHUSD', 1440)
        self.assertIsInstance(df, pd.DataFrame, "The result
           should be a pandas DataFrame.")
        self.assertFalse(df.empty, "The DataFrame should not
           be empty.")
```

Esta prueba unitaria asegura que los métodos de KrakenData funcionan como se espera, devolviendo los tipos de datos correctos y no produciendo resultados vacíos. Esto es crucial para validar la funcionalidad antes de desplegar el código en un entorno de producción. Si alguna de estas pruebas falla, indicará que hay un problema que necesita ser resuelto en el código de KrakenData.

Listing 6: Prueba unitaria para IndicatorCalculator.

```
import unittest
from ProyectoTradingDashboard import IndicatorCalculator
import pandas as pd
class TestIndicatorCalculator(unittest.TestCase):

    def setUp(self):
        self.df = pd.DataFrame({
            'close': [100, 102, 101, 103, 102, 101, 100, 99,
               98, 97, 96, 95, 94, 93, 92, 91, 90, 89, 88,
               87]
        })

    def test_calculate_bollinger_bands(self):
        df = IndicatorCalculator.calculate_bollinger_bands(
           self.df, window=5)
        self.assertIn('upper_band', df, "The DataFrame should
            contain 'upper_band'.")
```

```
        self.assertIn('lower_band', df, "The DataFrame should contain 'lower_band'.")
        self.assertIn('buy_signal', df, "The DataFrame should contain 'buy_signal'.")
        self.assertIn('sell_signal', df, "The DataFrame should contain 'sell_signal'.")

    def test_calculate_rsi(self):
        df = IndicatorCalculator.calculate_rsi(self.df, window=5)
        self.assertIn('rsi', df, "The DataFrame should contain 'rsi'.")

    def test_calculate_macd(self):
        df = IndicatorCalculator.calculate_macd(self.df, fast_period=5, slow_period=10, signal_period=3)
        self.assertIn('macd', df, "The DataFrame should contain 'macd'.")
        self.assertIn('macd_signal', df, "The DataFrame should contain 'macd_signal'.")
```

Esta prueba unitaria asegura que los métodos de IndicatorCalculator funcionan correctamente, añadiendo las columnas esperadas al DataFrame sin errores. Esto es crucial para validar que los cálculos de indicadores técnicos se realizan como se espera antes de utilizar estos métodos en un entorno de producción.

7. Ejecución del código

Para ejecutar el código del proyecto, sigue estos pasos:

1. Ejecutar en Visual Studio Code:

   ```
   ProyectoTradingDashboard.py
   pip install -r requirements.txt (Terminal)
   ```

2. Clona el repositorio y ejecuta:

   - **Clonar el repositorio**: El primer paso es clonar el repositorio del proyecto desde un repositorio remoto (como GitHub) a tu máquina local. Esto se hace generalmente con un comando como:

   ```
   git clone https://github.com/JSRP00/Python_ProyectoTradingDashboard
   ```

   - **Instalar dependencias**:

   ```
   pip install -r requirements.txt
   ```

   Este comando instala todas las dependencias necesarias para ejecutar el proyecto, que están listadas en el archivo `requirements.txt`.

- **Ejecutar el código**: Ejecuta el archivo principal del proyecto con el comando:

```
python main.py
```

  Este archivo contiene el código principal para iniciar la aplicación.

3. Facilitar los mecanismos para la reproducción del entorno virtual (ya sea con Poetry, Pipenv o pip):

```
# Para instalar las dependencias con Poetry
poetry install

# Para instalar las dependencias con Pipenv
pipenv install

# Para instalar las dependencias con pip
pip install -r requirements.txt
```

4. Utilizar Git y GitHub para la gestión del código:

   - Utiliza Git y GitHub para versionar y colaborar en el proyecto. Sigue buenas prácticas de desarrollo colaborativo, como hacer commits frecuentes y claros, utilizar ramas para nuevas funcionalidades, y hacer pull requests para revisión de código.

5. Abre en el navegador:

   - Después de ejecutar el código, debes abrir tu navegador y dirigirte a `http://127.0.0.1:8050/`. Este es un enlace local que usualmente se utiliza para acceder a aplicaciones web que están corriendo en tu máquina local. En este caso, probablemente estás utilizando un framework como Dash o Flask para desarrollar una aplicación web, y desde la dirección URL se puede hacer de manera interactiva.

7.1. Resumen

En resumen, estos pasos te guían a través de:

1. **Preparar tu entorno** al clonar el repositorio y instalar las dependencias.
2. **Ejecutar la aplicación** al iniciar el archivo principal de código.
3. **Acceder a la interfaz web** de la aplicación en tu navegador.
4. **Reproducir el entorno virtual** utilizando herramientas como Poetry, Pipenv o pip.
5. **Gestionar el proyecto** utilizando Git y GitHub para versionar y colaborar en el código.